TUPLES: FIXED RECORDS

A **tuple** is ordered and **cannot be changed** after it is created. Write it with `( )`.

1) Create tuples correctly

Example :

```
one_item = ("Python",)
not_a_tuple = ("Python")
empty_tuple = ()
print(one_item, type(one_item))
print(not_a_tuple, type(not_a_tuple))
print(empty_tuple, type(empty_tuple))
```

Output :

```
('Python',) <class 'tuple'>
Python <class 'str'>
() <class 'tuple'>
```

☆ REMEMBER

- A one-item tuple needs a comma: `("Python",)`. Without it, `("Python")` is just a string in brackets.

2) Access, slice and immutability

Example :

```
student = ("Priya", 19, "Data Science", 92, 87)
print(student[0], student[-1])
print(student[:2])
print(student[3:])
print(len(student), 100 in student)
new = student[:2] + ("Maths",) + student[3:]
print(new)
```

Output :

```
Priya 87
('Priya', 19)
(92, 87)
5 False
('Priya', 19, 'Maths', 92, 87)
```

☆ WHY AN ERROR?

- `student[2] = 90` raises `TypeError: 'tuple' object does not support item assignment.`
- Replace, append and remove are impossible: build a **new tuple** instead, as `new` above.

3) Methods and built-in functions

Each example uses `marks = (85, 92, 78, 92, 88)`.

Tool	What it does	Example	Result
`.count(x) .index(x)`	times x appears / index of first x	`marks.count(92), marks.index(92)`	2, 1
`len() sum()`	number of items / total	`len(marks), sum(marks)`	5, 435
`min() max()`	smallest / largest	`min(marks), max(marks)`	78, 92

UNPACKING & DICTIONARIES

Unpack a tuple into variables, then store **key: value** pairs in a **dictionary**.

1) Tuple unpacking

Example :

```
name, age, marks = ("Amit", 20, 78)
print(name, age, marks)
first, *middle, last = (10, 20, 30, 40, 50)
print(first, middle, last)
```

Output :

```
Amit 20 78
10 [20, 30, 40] 50
```

*variables must match the items, unless one starts with * (it collects the rest)*

2) What is a dictionary?

Example :

```
student = {"name": "Rahul", "age": 18, "marks": 85}
print(student)
print(student["name"], student["marks"])
```

Output :

```
{'name': 'Rahul', 'age': 18, 'marks': 85}
Rahul 85
```

Keys are unique and hashable, a dict is mutable and keeps insertion order

3) Access values: [ ] and get()

Each example uses `student = {"name": "Priya", "marks": 92}`.

Syntax	Example	Result
`[key]`	`student["marks"]`	92
`[key]` (missing)	`student["email"]`	KeyError
`.get(key)`	`student.get("email")`	None
`.get(key, default)`	`student.get("email", "Not provided")`	'Not provided'

4) Add and update entries

Example :

```
student = {"name": "Amit", "marks": 78}
student["course"] = "Python"
student["marks"] = 82
print(student)
student.update({"age": 20, "city": "Pune"})
print(student["age"], student["city"])
```

Output :

```
{'name': 'Amit', 'marks': 82, 'course': 'Python'}
20 Pune
```

a new Key adds an entry, an existing Key updates it. Keys are case-sensitive

Dictionary Methods & Loops

Remove entries, view keys and values, loop through a dictionary, and nest dictionaries.

1) Remove entries

Each example starts from `student = {"name": "Rahul", "age": 18, "marks": 85}`.

Method	What it does	Result
`.pop("marks")`	removes the key, returns its value	`returns 85`
`.popitem()`	removes the last inserted pair	`returns ('marks', 85)`
`del student["age"]`	deletes a specific key	`{'name': 'Rahul', 'marks': 85}`
`.clear()`	removes all entries	`{}`

2) Views and loops

Example :

```
student = {"name": "Neha", "marks": 94}
print(student.keys())
print(student.values())
print(student.items())
print("name" in student, "email" in student)
for key, value in student.items():
    print(key, "->", value)
```

Output :

```
dict_keys(['name', 'marks'])
dict_values(['Neha', 94])
dict_items([('name', 'Neha'), ('marks', 94)])
True False
name -> Neha
marks -> 94
```

for Key in d: loops over Keys; d.values() over values; d.items() over pairs

3) Nested dictionaries and tuple keys

Example :

```
students = {101: {"name": "Rahul", "marks": 85},
            102: {"name": "Priya", "marks": 92}}
print(students[102]["name"], students[102]["marks"])
locations = {(18.52, 73.86): "Pune"}
print(locations[(18.52, 73.86)])
```

Output :

```
Priya 92
Pune
```

4) Tuple vs dictionary

Feature	Tuple	Dictionary
Stores	ordered values	key-value pairs
Access	by index: `record[0]`	by key: `student["name"]`
Mutable?	No	Yes
Good for	fixed records, coordinates	named fields, lookup tables

Mistakes & Practice

Spot the bug, then try six short tasks first. Compare with the model solutions.

1) Spot the bug

Buggy line	Problem	Fix
`x = ("Python")`	this is a str, not a tuple	`x = ("Python",)`
`marks[0] = 90` (tuple)	TypeError: tuples cannot be changed	build a new tuple
`student["email"]` (missing)	KeyError: the key does not exist	`student.get("email")`
`a, b = (1, 2, 3)`	ValueError: too many values to unpack	`a, b, c = (1, 2, 3)`
`student["Name"]`	KeyError: "Name" is not "name"	`student["name"]`

2) Practice tasks and solutions

Task 1 Five days: first and last.

Solution :

```
days = ("Mon", "Tue", "Wed", "Thu", "Fri")
print(days[0], days[-1])
```

```
Mon Fri
```

Task 2 One-item tuple; count 90.

Solution :

```
one = ("Python",)
print(type(one))
print((90, 75, 90, 88, 90).count(90))
```

```
<class 'tuple'>
3
```

Task 3 Unpack a tuple.

Solution :

```
name, age, marks = ("Neha", 17, 94)
print(name, age, marks)
```

```
Neha 17 94
```

Task 4 Create a dict; print the name.

Solution :

```
student = {"name": "Priya", "age": 21,
           "course": "Python", "marks": 85}
print(student["name"])
```

```
Priya
```

Task 5 Get default; add grade; update marks.

Solution :

```
student = {"name": "Priya", "marks": 85}
print(student.get("email", "Not provided"))
student["grade"] = "A"
student["marks"] = 90
print(student)
```

```
Not provided
{'name': 'Priya', 'marks': 90, 'grade': 'A'}
```

Task 6 Loop through keys and values.

Solution :

```
student = {"name": "Priya", "marks": 90}
for key, value in student.items():
    print(key, value)
```

```
name Priya
marks 90
```

Quick check

Which syntax makes a one-item tuple? **("Python",)** Can a tuple item be reassigned? **No**